

REST API Vulnerabilities — Interview Prep Notes

Scope: common REST API-specific vuln classes NOT already covered (IDOR/BOLA, SSRF, SQLi, XSS, CSRF, CORS, Session, Path Traversal done separately). Covers: **Mass Assignment / Excessive Data Exposure (BOPLA)**, **BFLA**, **Unrestricted Resource Consumption (Rate Limiting)**, **Improper Inventory Mgmt (Shadow/Zombie APIs)**, **API Security Misconfiguration**.

Framework: OWASP API Security Top 10 **2023** edition used throughout (not 2019 — note the merge/renumber for interview accuracy).

0. Shared REST API Attack Surface (applies to all below — not repeated per vuln)

- Endpoints: CRUD routes, nested resources (`/users/{id}/orders/{id}`)
 - Params: path params, query params, request body (JSON/XML/form), headers (custom `X-` headers)
 - Auth: JWT/OAuth tokens, API keys, session cookies
 - API docs: Swagger/OpenAPI spec, Postman collections, GraphQL introspection (if hybrid)
 - Versioning: `/v1/`, `/v2/`, deprecated/beta/internal routes
 - Entry points: mobile app API calls (proxy traffic), webhooks, third-party integrations, admin/internal APIs exposed externally
-

1. Mass Assignment + Excessive Data Exposure → BOPLA (Broken Object Property Level Authorization)

Q1. Describe

- **Mass Assignment**: client sends extra JSON fields (`role`, `isAdmin`, `balance`) → server auto-binds request body to object/ORM model without allow-list → attacker modifies fields not intended to be user-writable.
- **Excessive Data Exposure**: server returns full object (all fields) → client-side filtering only → attacker inspects raw response to see fields UI hides (password hash, internal IDs, PII).
- 2023 OWASP merged both under **API3: Broken Object Property Level Authorization** — property-level read (exposure) vs write (mass assignment) issue.

Q2. OWASP + CWE

- OWASP API Top 10 2023: **API3:2023 - Broken Object Property Level Authorization**

- CWE-915 (Improperly Controlled Modification of Dynamically-Determined Object Attributes) — mass assignment
- CWE-213 (Exposure of Sensitive Info Due to Incompatible Policies) — excessive exposure
- Legacy 2019 mapping: API6:2019 (Mass Assignment), API3:2019 (Excessive Data Exposure) — mention both if asked "old vs new"

Q3. Variants

- Object property mass assignment (privilege field, price field)
- Array/nested object injection (add unauthorized items to array)
- Excessive exposure via GET (full user object incl. hash/internal flags)
- Excessive exposure via error messages/stack traces (adjacent to misconfig)
- GraphQL equivalent: over-fetching via query (separate topic)

Q4. Attack surface / entry points

- POST/PUT/PATCH bodies (registration, profile update, checkout, admin create)
- Any endpoint using auto-binding ORM/serializer (Django REST `ModelSerializer`, Rails `strong_parameters` misuse, Spring `@ModelAttribute`, Node `Object.assign(user, req.body)`)
- GET responses on `/users/{id}`, `/orders/{id}` returning full DB row

Q5. Testing workflow

1. **Baseline diff** — verify: normal request/response shape
 - Manual: capture legit request, note writable fields per docs
 - Burp: Proxy → capture request → send to Repeater
 - ZAP: HUD/History → send to Manual Request Editor
 - Nuclei: not ideal for discovery phase (no baseline)
2. **Field enumeration** — verify: which extra fields server accepts
 - Manual: append candidate fields (`role`, `isAdmin`, `is_verified`, `price`, `discount`, `user_id`, `balance`, `status`) to body, observe response/DB state change
 - Burp: Repeater + Intruder (sniper, wordlist of common privileged field names) → param mining extension (Param Miner)
 - ZAP: Fuzzer with field-name wordlist injected into JSON body
 - Nuclei: custom template with fuzz block against known field-name list; existing templates limited (mostly generic)
 - Payloads: `{"role":"admin"}`, `{"isAdmin":true}`, `{"price":0}`, `{"user_id":<victim_id>}`, `{"verified":true}`
3. **Response field audit** — verify: response leaks fields not shown in UI
 - Manual: diff full JSON response vs rendered UI fields
 - Burp: Repeater response tab, search for `password`, `hash`, `token`, `ssn`, `internal_`
 - ZAP: Response viewer + regex search (Alert Filters)

- Nuclei: existing templates for common leaked-field patterns (e.g., `exposed-panels`, custom regex extractor template)

4. **Confirm impact** — verify: field change persists / grants privilege

- Manual: re-authenticate/re-fetch resource to confirm state changed
- Burp/ZAP: repeat GET after PATCH, compare
- Nuclei: N/A (needs stateful chain)

Q6. Evidence

- Request/response pair showing injected field accepted (before/after state)
- Screenshot of privilege change (e.g., role now `admin` in subsequent GET)
- Response body showing leaked sensitive field (hash, token, PII) with redaction in report

Q7. PoC

- Register/update account with `{"role": "admin"}` in body → subsequent login/GET shows admin role → access admin-only endpoint successfully. Full loop: inject → verify persisted → demonstrate privileged action.

Q8. Attack chains

- Mass assignment → privilege escalation → BFLA exploitation (access admin endpoints)
- Excessive exposure → leaked internal ID/token → IDOR chain on another endpoint
- Price/discount mass assignment → business logic bypass (free checkout)
- Leaked password hash → offline cracking → account takeover

Q9. False positives

- Field accepted but server-side re-validates/overwrites before persist (no real impact) — must confirm via re-fetch
- Field present in response but is non-sensitive/intentionally public
- Rejected with 400/422 due to schema validation ≠ vulnerable

Q10. Prerequisites

- No allow-list/DTO enforcing writable fields (relies on blacklist or none)
- Server-side auto-binding (ORM-to-request-body mapping) without explicit field mapping
- No property-level authorization check post-deserialization

Q11. Attacker achieves

- Privilege escalation (self → admin)
- Data tampering (price, balance, ownership fields)
- PII/credential disclosure via response

Q12. CIA Impact

Variant	Confidentiality	Integrity	Availability	CVSS v3.1 Vector (typical)
Mass Assignment (priv esc)	Low	High	None	AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:N
Mass Assignment (financial field)	Low	High	Low	AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:L
Excessive Data Exposure (PII/hash)	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N

Q13. CVSS severity

- Mass assignment to admin role: **Critical (9.0–9.9)**, Scope Changed if it escalates beyond the API's own authorization domain
- Excessive data exposure (PII): **High (7.0–8.9)**

Q14. Business impact (financial app)

- Balance/limit tampering → direct monetary loss
- Role escalation → fraud, unauthorized fund transfer approval
- PII/account data leak → PCI-DSS/GDPR breach notification, regulatory fine, reputational damage
- Mass assignment on KYC `verified` flag → AML/compliance bypass

Q15. Detection signals

- Unexpected fields in request body accepted (200/201 with no rejection)
- Response payload size/field count > documented schema
- WAF/logs: repeated requests with schema-violating extra keys
- SAST: direct `req.body` → model save without DTO/serializer allow-list

Q16. Root cause (code)

```
// Vulnerable (Node/Mongoose)

const user = await User.findById(id);
Object.assign(user, req.body); // blind bind - role, isAdmin included
await user.save();

// Vulnerable (Django REST, fields = '__all__')
class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = '__all__' // exposes + allows write on all fields
```

- Root cause: object mapped from client input to persistence/DB layer with no explicit allow-list of writable/readable fields; trust boundary between wire format and domain model missing.

Q17. Remediation

- Explicit **DTO/allow-list** per endpoint (only expected fields bound)
- Serializer field control: `fields = ['name', 'email']` not `__all__`; separate **read** vs **write** serializers
- Reject unknown fields (`additionalProperties: false` in JSON schema validation)
- Property-level authorization check post-bind before persist (role changes require separate privileged endpoint)
- Response DTOs strip internal fields (never serialize DB row directly)

Q18. Compensating controls

- WAF rule blocking known sensitive field names in POST/PUT/PATCH bodies
- API gateway schema validation (OpenAPI strict mode) rejecting unknown properties
- Logging/alerting on privileged-field presence in requests from non-admin roles
- Read-only DB replica field masking for sensitive columns in API layer

Q19. Recurrence indicates

- No standard DTO/serializer pattern enforced org-wide → systemic SDLC gap
- Missing "generate response/request schema from allow-list" as dev standard
- Points to absent API design review / schema-first development

Q20. Upstream improvements

- Semgrep rule: flag `Object.assign(model, req.body)`, `fields = '__all__'`, `@ModelAttribute` without `@InitBinder` allow-list
- CodeQL: dataflow from request body param → ORM save call without intermediate DTO
- Framework-level: enforce OpenAPI schema validation middleware (reject `additionalProperties`)
- Design checklist: "every endpoint has explicit request/response schema reviewed for sensitive fields"

Q21. Sample triage response

Confirmed BOPLA (Mass Assignment) on `PATCH /api/users/{id}`. Injecting `"role": "admin"` in request body persists and grants admin privileges on subsequent login, confirmed via access to `/api/admin/dashboard`. CVSS 9.1 (Critical). Evidence attached (req/resp pair, before/after role state). Recommend immediate allow-list fix on serializer.

Q22. Sample dev remediation note

Endpoint `PATCH /api/users/{id}` binds full request body to `User` model. Replace `Object.assign(user, req.body)` with explicit allow-listed fields (`name`, `email`, `phone` only). Role/permission changes must go through separate admin-only endpoint with its own authz check. Add JSON schema validation (`additionalProperties:false`) at gateway/middleware layer as defense-in-depth.

Q23. Real-world reference

- Numerous HackerOne mass-assignment reports on role/price fields (common in bug bounty triage patterns); GitHub's 2012 mass-assignment public key incident (Rails, pre `strong_parameters`) is the canonical historical case.

2. BFLA — Broken Function Level Authorization

Q1. Describe

- User with lower privilege calls an API function/endpoint intended for higher-privilege role (e.g., regular user calls `DELETE /admin/users/{id}`) — authz check missing/misconfigured at the **function/endpoint** level (vs BOLA which is object-instance level).

Q2. OWASP + CWE

- **API5:2023 - Broken Function Level Authorization**
- CWE-285 (Improper Authorization), CWE-862 (Missing Authorization), CWE-863 (Incorrect Authorization)

Q3. Variants

- Horizontal function bypass (user role calling other-user-role functions of same level — rare, usually vertical)
- Vertical: low-priv → admin function
- HTTP verb tampering (GET allowed, but same route's DELETE/PUT unchecked)
- Hidden/undocumented admin endpoints reachable without role check

Q4. Attack surface

- Admin/internal endpoints (`/admin/*`, `/internal/*`, `/api/v1/manage/*`)
- Same resource path, different verb (`GET /orders/{id}` protected, `DELETE /orders/{id}` not)
- Endpoints discovered via JS bundle, Swagger, mobile app reverse-engineering, wordlists (ffuf/gobuster)

Q5. Testing workflow

1. **Endpoint inventory by role** — verify: map which endpoints exist per role
 - Manual: crawl app as each role, note accessible functions
 - Burp: Site map compare across roles (Target tab), or Authorize/AuthMatrix extension
 - ZAP: Access Control Testing add-on (context-based roles)
 - Nuclei: N/A for discovery
2. **Cross-role replay** — verify: low-priv token can call high-priv endpoint
 - Manual: capture admin request, replace token/cookie with low-priv session, resend
 - Burp: Repeater — swap Authorization header, or Authorize extension auto-diffs authenticated-as-lower-role responses
 - ZAP: Replace header via Replacer + manual resend, or Access Control add-on automation
 - Nuclei: custom template can automate known-endpoint + low-priv-token request, check for 200 vs 401/403
 - Payloads: swap JWT to low-priv, remove `Authorization` header entirely (test unauthenticated), verb tampering (`GET` → `DELETE` / `PUT` / `PATCH` on same path)
3. **Verb/method fuzzing** — verify: alternate HTTP verbs bypass check
 - Manual: try all verbs on protected route
 - Burp: Intruder with verb list in request line
 - ZAP: Fuzzer on method
 - Nuclei: template looping methods against endpoint list

Q6. Evidence

- Two requests side by side: admin token (200 + result) vs low-priv token (200 + same result) — should have been 401/403
- Response body confirming action executed (e.g., resource deleted, admin data returned)

Q7. PoC

- Low-priv user token → `DELETE /api/admin/users/5` → 200 OK, user 5 deleted, confirmed via subsequent admin GET showing user gone.

Q8. Attack chains

- BFLA → full admin takeover of application
- BFLA + Mass Assignment → self-elevate role then use now-authorized-looking token on admin functions
- BFLA on user management → mass account takeover

Q9. False positives

- Endpoint returns 200 but with empty/error payload (soft-fail, not real bypass)

- Function is intentionally public (no auth required by design)
- Client-side-only redirect prevents UI access, but check must confirm server-side enforcement (this is actually a true positive if server doesn't check)

Q10. Prerequisites

- Authz check missing/incomplete at endpoint/controller level (decorator/middleware not applied uniformly)
- Reliance on client-side role hiding (UI hides button, API doesn't check)
- Shared controller logic without per-action role guard

Q11. Attacker achieves

- Execute admin functions (user mgmt, config change, data deletion) as low-priv user

Q12. CIA Impact

Variant	Confidentiality	Integrity	Availability	CVSS Vector (typical)
BFLA - read-only admin data	High	None	None	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
BFLA - admin write/delete	Low	High	High	AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:H

Q13. CVSS severity

- Typically **Critical (9.0+)** when leads to full admin function access, Scope Changed if crosses privilege boundary of app's security model

Q14. Business impact (financial)

- Unauthorized fund transfer/approval functions triggered by non-privileged user
- Regulatory: violates least-privilege controls mandated by PCI-DSS/SOX in financial systems
- Mass account manipulation → systemic fraud risk, forced audit/incident disclosure

Q15. Detection signals

- Low-priv session successfully calling privileged endpoint (200 vs expected 403) in access logs
- Anomalous role-token vs endpoint-sensitivity mismatch in SIEM correlation
- Sudden spike in admin-endpoint calls from non-admin user IDs

Q16. Root cause (code)

```
// Vulnerable (Express) - route registered without role middleware
router.delete('/admin/users/:id', deleteUser); // missing requireAdmin()

// Vulnerable (Spring) - inconsistent @PreAuthorize
@DeleteMapping("/admin/users/{id}")
public void deleteUser(@PathVariable id) { ... } // no @PreAuthorize("hasRole('ADMIN')")
```

- Root cause: authorization enforced ad-hoc per route/controller instead of centrally, so new/edited endpoints silently miss the check; no default-deny posture.

Q17. Remediation

- **Default-deny** middleware: every route requires explicit role annotation to be reachable
- Centralized RBAC/ABAC middleware applied globally, allow-list public routes explicitly
- Consistent decorator/annotation enforcement (`@PreAuthorize`, `requireRole()`) reviewed in code review checklist
- Server-side role check independent of client-supplied role claims (validate against DB/session, not just JWT claim if JWT not signed/verified properly)

Q18. Compensating controls

- API gateway-level route-to-role mapping enforced independently of app code
- WAF/gateway blocking non-admin-network-source calls to `/admin/*` paths
- Monitoring/alerting on role-endpoint mismatch access attempts

Q19. Recurrence indicates

- No centralized authorization framework — authz treated as afterthought per-endpoint
- Missing security review gate for new endpoint additions in SDLC
- Points to needed RBAC middleware refactor + endpoint authz test coverage in CI

Q20. Upstream improvements

- Semgrep/CodeQL rule: flag routes/controllers missing role-check annotation/middleware
- Automated authz test suite: for every endpoint, test with each role, assert expected status (contract test)
- Framework default: deny-by-default route registration requiring explicit `@Public` opt-out
- Design checklist: RBAC matrix reviewed and versioned alongside API spec

Q21. Sample triage response

Confirmed BFLA on `DELETE /api/admin/users/{id}`. Standard user token successfully invokes admin-only delete function (200 OK, resource removed), verified server-side enforcement absent — client-side role hiding only. CVSS 9.1 Critical. PoC and role-comparison evidence attached.

Q22. Sample dev remediation note

`DELETE /api/admin/users/{id}` lacks server-side role check; only UI hides the action. Add `requireRole('admin')` middleware (or `@PreAuthorize("hasRole('ADMIN')")`) consistent with other admin routes. Recommend adding automated RBAC contract tests covering all roles per endpoint to CI to prevent regression.

Q23. Real-world reference

- Common recurring class in H1/Bugcrowd API-focused programs (e.g., admin panel functions reachable via direct API call bypassing UI role gating) — frequently reported against SaaS admin consoles.

3. Unrestricted Resource Consumption (Rate Limiting / DoS)

Q1. Describe

- API lacks limits on resource-intensive requests: request rate, payload size, pagination size, execution time, or concurrent connections — allows attacker to exhaust compute/DB/bandwidth or brute-force auth/OTP.

Q2. OWASP + CWE

- **API4:2023 - Unrestricted Resource Consumption**
- CWE-770 (Allocation of Resources Without Limits), CWE-799 (Improper Control of Interaction Frequency), CWE-307 (Improper Restriction of Excessive Auth Attempts)

Q3. Variants

- Missing rate limiting → brute force (login, OTP, password reset)
- Missing pagination limit → large `limit=` param causes DB overload
- Large payload/file upload with no size cap
- Expensive query params (unbounded search/filter/regex) → CPU exhaustion
- No concurrent-request cap → connection pool exhaustion
- Missing timeout on long-running requests

Q4. Attack surface

- Login, OTP/2FA verify, password reset, signup endpoints
- Search/filter/list endpoints with `limit` / `page_size` params
- File upload endpoints
- Any endpoint calling expensive downstream (regex, image processing, PDF gen, external API call)

Q5. Testing workflow

1. **Rate limit presence check** — verify: is throttling enforced at all
 - Manual: send N rapid repeated requests, observe if blocked/throttled (429)
 - Burp: Intruder (sniper, high thread count) against endpoint, monitor response codes/timing
 - ZAP: Fuzzer with high request count, or Active Scan's DoS-related rules
 - Nuclei: existing `rate-limit` detection templates available; workflow-style template sending burst requests and checking absence of 429
 - Payloads: N/A — volume-based; note request/sec threshold tested
2. **Bypass technique check** — verify: can rate limit be bypassed via header/IP rotation
 - Manual: rotate `X-Forwarded-For`, `X-Real-IP`, user-agent, or use multiple accounts/tokens
 - Burp: Intruder with header payload list rotating IP-spoof headers
 - ZAP: Fuzzer with header rotation
 - Nuclei: custom template testing header-based bypass
 - Payloads: `X-Forwarded-For: <random_ip>`, `X-Originating-IP`, alternate session tokens
3. **Resource-limit check (pagination/payload)** — verify: oversized params accepted
 - Manual: send `?limit=999999`, huge JSON body, large file upload
 - Burp: Repeater with modified params, observe response time/server load proxy signals
 - ZAP: Manual request editor
 - Nuclei: N/A typically (needs load observation)
 - Payloads: `limit=100000`, deeply nested JSON, oversized multipart file, regex-catastrophic-backtracking string in search field (ReDoS)

Q6. Evidence

- Screenshot/log of successful brute force (e.g., OTP guessed within N attempts, no logout)
- Response timing showing degraded server response under load test
- 429 absent across sustained request burst (proxy history export)

Q7. PoC

- Automated script sends 1000 login attempts with different passwords within short window, no 429/lockout triggered, account compromised via brute force — demonstrates lack of rate limit with real credential compromise outcome.

Q8. Attack chains

- No rate limit on OTP → account takeover
- No rate limit on password reset token → brute force reset token → ATO
- Resource exhaustion → DoS → business availability impact
- Enumeration (no rate limit + verbose error) → user enumeration → targeted credential stuffing

Q9. False positives

- 429 present but threshold high (not "no limit", just generous) — note actual threshold, may still be a finding if too permissive
- Rate limit exists per-IP but trivially bypassed — still a valid finding, not FP, but distinguish "present but weak" from "absent"
- WAF-level blocking (not app-level) — check both layers before concluding absent

Q10. Prerequisites

- No throttling middleware/gateway policy configured
- No CAPTCHA/step-up on sensitive flows (OTP, login)
- No account lockout policy

Q11. Attacker achieves

- Brute force credentials/OTP/tokens → ATO
- Denial of service → downtime, SLA breach
- Cost amplification (cloud billing abuse on metered APIs)

Q12. CIA Impact

Variant	Confidentiality	Integrity	Availability	CVSS Vector (typical)
Brute force (OTP/login, no rate limit)	High	Low	None	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Resource exhaustion / DoS	None	None	High	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
ReDoS (catastrophic regex)	None	None	High	AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H

Q13. CVSS severity

- Brute-forceable auth/OTP: **High (7.5–8.8)**
- Pure DoS: **High (7.5)** typically (AC:L, no auth) unless amplification is extreme (then Critical)

Q14. Business impact (financial)

- OTP brute force → unauthorized fund transfer/account access → direct financial loss + regulatory (RBI/PCI-DSS) reporting obligation
- DoS on trading/payment API → SLA breach, revenue loss during downtime, reputational damage
- Cloud cost abuse on pay-per-call APIs → direct financial cost

Q15. Detection signals

- Spike in requests/sec from single IP/account in logs
- High volume of 401/403 (failed auth) from same source
- APM: latency/CPU spike correlated with specific endpoint traffic burst

Q16. Root cause (code)

```
// Vulnerable - no throttle wrapper
app.post('/login', async (req,res) => {
  const user = await verifyCredentials(req.body); // unlimited attempts
});

// Vulnerable - unbounded pagination
const results = await db.query(`SELECT * FROM items LIMIT ${req.query.limit}`); // no
max cap
```

- Root cause: no centralized rate-limit/throttle enforcement layer; input controlling resource size (limit, file size) not bounded server-side.

Q17. Remediation

- Rate limiting middleware (token bucket/sliding window) at gateway + app layer, per-IP AND per-account
- Account lockout / exponential backoff on repeated auth failures
- Enforce max `limit` / `page_size` server-side regardless of client input
- Request size caps (body size, file upload size) at gateway
- Timeouts on downstream calls; circuit breakers
- CAPTCHA/step-up MFA after N failed attempts on sensitive endpoints

Q18. Compensating controls

- WAF/CDN-level rate limiting (Cloudflare, AWS WAF rate rules) as stopgap
- API gateway quota policies (Kong, Apigee, AWS API Gateway usage plans)
- Alerting-based manual response (SOC blocks IP on threshold breach) until fix deployed

Q19. Recurrence indicates

- No API gateway/throttling standard adopted across services — systemic infra gap
- Missing non-functional requirement (rate limiting) in API design template
- Points to need for shared middleware library, not per-team reinvention

Q20. Upstream improvements

- Standard shared rate-limit middleware library mandated for all new endpoints (mirrors your shared-wrapper strategy for IDOR)
- API gateway policy enforced by default for all routes (opt-out not opt-in)
- Design checklist item: "rate limit + max payload size defined in OpenAPI spec `x-rate-limit` extension"
- Load/DoS test cases added to CI performance test suite

Q21. Sample triage response

Confirmed missing rate limiting on `POST /api/auth/otp-verify`. 4-digit OTP brute-forceable in ~10k requests, completed in <5 min, no lockout/429 observed. Account takeover achieved in PoC. CVSS 8.1 (High). Recommend immediate rate limit + lockout policy.

Q22. Sample dev remediation note

`POST /api/auth/otp-verify` has no attempt limiting. Add rate limiting (max 5 attempts per 15 min per account+IP) using existing `@RateLimit` middleware used on `/login`. Add exponential lockout after threshold. Ensure gateway-level policy also applied as defense-in-depth.

Q23. Real-world reference

- Numerous bug bounty reports on OTP/2FA brute force due to missing rate limiting (common H1 finding category across fintech programs); T-Mobile API breach (2023) partly attributed to lack of rate limiting on an exposed API enabling mass data scraping.

4. Improper Inventory Management (Shadow / Zombie / Undocumented APIs)

Q1. Describe

- Organization loses track of API surface: old/deprecated versions still live (zombie), undocumented/internal APIs exposed externally (shadow), or dev/staging APIs reachable in prod — these often lack current security controls applied to documented/maintained endpoints.

Q2. OWASP + CWE

- **API9:2023 - Improper Inventory Management**
- CWE-1059 (Insufficient Technical Documentation) — closest mapping; often paired with CWE-284 (Improper Access Control) since shadow APIs typically lack current controls

Q3. Variants

- Shadow API (never documented, found via traffic analysis/JS)
- Zombie API (deprecated version `/v1/` still live after `/v2/` released, missing new controls)
- Debug/internal APIs exposed to internet (staging, admin-only meant for VPN)
- Third-party/partner API integration with excess scope exposed

Q4. Attack surface

- Old API versions (`/v1/` , `/api-old/`)
- JS bundle-referenced endpoints not in Swagger
- Subdomains: `api-dev.` , `staging-api.` , `internal-api.`
- Mobile app decompiled endpoints not present in web app docs

Q5. Testing workflow

1. **Discovery / inventory build** — verify: full list of live API endpoints/versions
 - Manual: review JS bundles, mobile app APK/IPA strings, robots.txt, sitemap, Wayback/gau/waybackurls output (your recon pipeline)
 - Burp: Content discovery, site map passive crawl, JS Link Finder extension
 - ZAP: Spider + AJAX spider, Forced Browse (DirBuster)
 - Nuclei: `technologies` / `exposures` template tags for exposed Swagger/API docs, subdomain enum templates
 - Payloads: version bruteforce (`/v1/` , `/v0/` , `/api/internal/` , `/api/debug/`), wordlists (SecLists `api/`)
2. **Version/deprecated endpoint testing** — verify: old version still functional & under-protected
 - Manual: replay current-version auth token against old-version endpoint path
 - Burp: Repeater — swap path version, keep token, observe response
 - ZAP: Manual editor path swap
 - Nuclei: custom template checking known old-version paths return 200
3. **Exposure confirmation** — verify: endpoint reachable without intended network restriction (should be internal-only)
 - Manual: access from external IP, check if internal-only marker (`X-Internal`) enforced or just documented convention
 - Burp/ZAP: direct request from external network
 - Nuclei: exposure-check templates for `/actuator` , `/swagger-ui` , `/graphql` (if hybrid), `/debug` , `/.env` on API hosts

Q6. Evidence

- List of discovered undocumented/deprecated endpoints with response codes
- Comparison showing old version lacks current security header/rate-limit/authz control present on current version
- Screenshot of internal endpoint reachable externally

Q7. PoC

- `/api/v1/users` (deprecated, undocumented in current API spec) still returns full user data with weaker auth (no MFA-bound token check present on `/v2/`) — demonstrates bypass of newer controls via old version.

Q8. Attack chains

- Shadow API discovery → BOLA/BFLA on undocumented endpoint (no monitoring/WAF coverage)
- Zombie API → bypass newer rate-limit/authz added only to current version
- Debug endpoint exposure → info disclosure → further recon for chained attacks

Q9. False positives

- Old version returns 410 Gone / properly deprecated with same controls — not vulnerable, just discoverable
- "Undocumented" endpoint that is intentionally public and equally protected

Q10. Prerequisites

- No API lifecycle/decommissioning process
- No centralized API gateway enforcing uniform policy across versions
- No automated inventory/discovery process (relies on manual documentation)

Q11. Attacker achieves

- Access to under-protected legacy functionality
- Bypass of controls added only to current-gen API (WAF rules, rate limits, authz)
- Broader recon surface for chaining other vulns

Q12. CIA Impact

Variant	Confidentiality	Integrity	Availability	CVSS Vector (typical)
Shadow API data exposure	High	Low	None	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Zombie API bypassing new controls	Low	High	None	AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:H/A:N

Q13. CVSS severity

- Varies widely by what's exposed (Medium–Critical); treat like the underlying vuln class found on the shadow endpoint (e.g., if it exposes BOLA, score as BOLA)

Q14. Business impact (financial)

- Deprecated payment/transaction API still live without current fraud controls → bypass of newer anti-fraud logic
- Regulatory: unmanaged API inventory is a common PCI-DSS/audit finding (asset management control failure)
- Reputational: silent data exposure via forgotten endpoint often surfaces via breach disclosure, not internal detection

Q15. Detection signals

- Traffic to undocumented paths in gateway/WAF logs (paths not in OpenAPI spec)
- Old version usage metrics not zero post-deprecation deadline
- New subdomain/host discovered via cert transparency logs not in asset inventory

Q16. Root cause (code / process)

- Not typically a single code snippet — root cause is **process**: no automated spec-to-gateway sync, deprecated routes not removed/decommissioned in code, feature flags left enabling old routers:

```
// Zombie route left mounted
app.use('/api/v1', legacyRouter); // meant to be removed post-migration, forgotten
app.use('/api/v2', currentRouter); // has new auth middleware; v1 doesn't
```

Q17. Remediation

- API lifecycle policy: formal deprecation → sunset header (`Sunset` , `Deprecation` HTTP headers) → hard removal timeline
- Central API gateway as single enforcement point (all versions get uniform authz/rate-limit regardless of app code)
- Automated inventory: OpenAPI spec as source of truth, CI check that all live routes are in spec (diff live traffic vs spec)
- Remove dead code/routers on decommission, not just "unlink from docs"

Q18. Compensating controls

- WAF rule blocking access to known deprecated paths from external IPs
- Network segmentation: internal-only endpoints enforced at network/gateway level, not just convention

- Periodic external attack-surface scanning (subdomain enum, JS analysis) as continuous control

Q19. Recurrence indicates

- No API governance/gateway strategy — decentralized, ad-hoc endpoint creation
- SDLC missing "asset inventory update" as part of deploy pipeline
- Points to need for API catalog/governance tooling (e.g., API gateway with mandatory registration)

Q20. Upstream improvements

- CI check: live-traffic-discovered routes must exist in OpenAPI spec (fail build if drift)
- Mandatory API gateway registration before any route reaches internet-facing ALB/ingress
- Linter/pipeline rule: routers/controllers flagged if not referenced in current spec version
- Design checklist: deprecation plan required at time of new version release, not after

Q21. Sample triage response

Confirmed zombie API: `/api/v1/transfer` remains live post-`/v2/` migration, missing MFA-step-up control added to v2. Same functionality accessible bypassing newer control. CVSS 8.2 (High) — treated as authz bypass equivalent. Recommend immediate decommission or control backport.

Q22. Sample dev remediation note

`/api/v1/*` router still mounted despite `/v2/` being the documented current version. Confirm no active clients depend on v1 (check gateway access logs), then remove `legacyRouter` mount and route entirely. If clients still depend on it, backport MFA/rate-limit middleware from v2 as interim measure and set `Sunset` header with removal date.

Q23. Real-world reference

- Peloton (2021) API bug bounty case: undocumented/insufficiently protected API endpoint exposed private user data (age, location, gender) without proper auth checks — widely cited shadow/under-protected API case in industry writeups.

5. API-Specific Security Misconfiguration

Q1. Describe

- Insecure default/verbose configuration specific to API layer: verbose error/stack traces, missing security headers, permissive CORS (see separate CORS notes), unnecessary HTTP methods enabled, exposed API docs/admin endpoints (Swagger UI, actuator), default credentials on API management

console, missing TLS enforcement.

Q2. OWASP + CWE

- **API8:2023 - Security Misconfiguration**
- CWE-16 (Configuration), CWE-209 (Information Exposure Through Error Message), CWE-548 (Exposure of Info Through Directory Listing)

Q3. Variants

- Verbose error responses (stack traces, DB errors, internal paths)
- Exposed Swagger/OpenAPI UI, actuator/health/debug endpoints in prod
- Missing security headers (`X-Content-Type-Options` , `Strict-Transport-Security`) — API-relevant subset
- Unnecessary HTTP methods enabled (TRACE, PUT on read-only resource) — check via `OPTIONS`
- Default/weak credentials on API gateway/management console
- Missing TLS / mixed HTTP+HTTPS acceptance

Q4. Attack surface

- `/swagger-ui` , `/api-docs` , `/actuator` , `/actuator/env` , `/actuator/heapdump` , `/.well-known/` , `/debug` , `/health` (verbose)
- Error-triggering inputs (malformed JSON, wrong content-type) → stack trace leak
- `OPTIONS` method response revealing allowed methods

Q5. Testing workflow

1. **Verbose error check** — verify: server leaks stack trace/internal info
 - Manual: send malformed JSON, wrong content-type, invalid enum value
 - Burp: Repeater — mutate content-type/body, inspect response
 - ZAP: Active Scan (Server-Side Info Leak rules)
 - Nuclei: existing templates for common stack-trace signatures / exposed error pages
 - Payloads: `{invalid json}` , `Content-Type: application/xml` on JSON-only endpoint, huge int overflow value
2. **Exposed management/docs endpoint check** — verify: internal tooling reachable externally
 - Manual: request known paths (`/actuator/env` , `/swagger-ui.html` , `/api-docs` , `/.env`)
 - Burp: Content discovery / Intruder with wordlist
 - ZAP: Forced Browse add-on
 - Nuclei: **existing templates strong here** — `exposed-panels` , `exposures/configs` , `spring-actuator` templates directly applicable
 - Payloads: SecLists `Discovery/Web-Content/api/` wordlist
3. **HTTP method audit** — verify: unnecessary/dangerous methods enabled

- Manual: send `OPTIONS`, then try `TRACE`, `PUT`, `DELETE` on endpoints that should be read-only
- Burp: Repeater method dropdown iterate
- ZAP: Manual editor
- Nuclei: template checking `TRACE` method enabled (XST-related)

Q6. Evidence

- Response body with stack trace / internal path / DB engine version
- Screenshot of accessible Swagger UI / actuator endpoint in production listing env vars or internal routes
- `OPTIONS` response listing unexpected methods, followed by successful call to one (e.g., `PUT` on presumed-read-only resource)

Q7. PoC

- Request malformed body to `/api/orders` → 500 response with full stack trace revealing framework version, DB connection string path, internal file paths — usable for targeted follow-on attacks (version-specific CVE lookup).

Q8. Attack chains

- Verbose error → tech stack fingerprint → targeted CVE exploitation
- Exposed actuator `/env` → leaked secrets/credentials → lateral access to DB/cloud resources
- Exposed Swagger → full undocumented endpoint map → feeds BOLA/BFLA/mass-assignment testing

Q9. False positives

- Generic error message without actual internal detail (just "Something went wrong") — not a finding
- Actuator endpoint present but properly access-controlled (401 on all sub-paths)
- Swagger UI present but intentionally public (documented public API) with no sensitive internal routes listed

Q10. Prerequisites

- Framework debug/dev mode left enabled in production
- No centralized error handler sanitizing responses
- Default framework paths not disabled/secured post-deployment

Q11. Attacker achieves

- Tech stack/version fingerprinting → targeted exploitation
- Credential/secret harvesting (actuator env, config exposure)
- Full API map reconstruction (feeds every other API vuln class)

Q12. CIA Impact

Variant	Confidentiality	Integrity	Availability	CVSS Vector (typical)
Verbose error / stack trace	Low	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C: L/I:N/A:N
Exposed actuator w/ secrets	High	Low	None	AV:N/AC:L/PR:N/UI:N/S:U/C: H/I:L/A:N
TRACE/method misconfig (XST)	Low	Low	None	AV:N/AC:L/PR:N/UI:N/S:U/C: L/I:L/A:N

Q13. CVSS severity

- Verbose error alone: **Low–Medium (4.0–5.9)**
- Exposed secrets via actuator/config: **High–Critical (7.5–9.8)** depending on what's leaked (DB creds = critical)

Q14. Business impact (financial)

- Leaked DB/cloud credentials via actuator → direct path to core banking/transaction DB compromise
- Stack traces revealing internal architecture aid targeted attacks on payment processing components
- Regulatory: misconfiguration is a top recurring PCI-DSS audit finding category

Q15. Detection signals

- 5xx responses with body size/content anomalies in logs
- WAF/logs showing requests to `/actuator/*`, `/swagger*`, `/.env` from external IPs
- Config management drift detection (prod config differs from hardened baseline)

Q16. Root cause (code/config)

```
// Vulnerable (Spring Boot) - actuator fully exposed
management.endpoints.web.exposure.include=*
// no management.endpoint.env.enabled=false, no security config on /actuator/**

// Vulnerable - default error handler
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.stack }); // leaks stack trace to client
});
```

- Root cause: production deployed with dev/debug defaults; no environment-specific hardening step in deployment pipeline; centralized sanitized error handler absent.

Q17. Remediation

- Disable/secure actuator, debug, and docs endpoints in production (`management.endpoints.web.exposure.include=health` only, auth-protect rest)
- Centralized error handler returning generic message to client, full detail logged server-side only
- Explicit method allow-list per route (405 for unsupported methods), disable `TRACE`
- Environment-specific config hardening as mandatory deployment gate (checklist/automated scan pre-prod→prod promotion)

Q18. Compensating controls

- WAF rules blocking `/actuator/*`, `/.env`, `/swagger*` from external traffic
- Network-level restriction: management ports/paths only reachable from internal network/VPN
- Automated config scanning (e.g., Nuclei misconfig templates) in CI/CD against staging before prod deploy

Q19. Recurrence indicates

- No standardized secure deployment baseline/hardening checklist across services
- Dev-to-prod config promotion process lacks environment-specific override enforcement
- Points to need for infra-as-code hardened templates + pre-deploy config scan gate

Q20. Upstream improvements

- CI/CD gate: automated Nuclei/misconfig scan against staging before prod promotion
- Infra-as-code baseline templates with actuator/debug disabled by default, environment-parameterized
- Centralized error-handling library mandated org-wide (no ad-hoc `err.stack` responses)
- Design checklist: "production config diff reviewed against hardening baseline" pre-release

Q21. Sample triage response

Confirmed Security Misconfiguration: `/actuator/env` publicly accessible on production API host, exposing DB credentials and internal service URLs. CVSS 9.1 (Critical) — direct credential exposure. Recommend immediate access restriction/endpoint disablement.

Q22. Sample dev remediation note

`/actuator/env` and related actuator sub-paths exposed without authentication in prod. Set `management.endpoints.web.exposure.include=health,info` only, and add Spring Security rule requiring `ROLE_ADMIN` for remaining actuator paths. Rotate any credentials found in the exposed env dump immediately (assume compromised).

Q23. Real-world reference

- Widespread class of findings: exposed Spring Boot Actuator endpoints leaking credentials is one of the most frequently reported misconfiguration issues in enterprise bug bounty programs (recurring H1/Bugcrowd category); First American Financial Corp 2019 breach involved improper access control/misconfiguration exposing millions of documents via predictable/unauthenticated document API.

Quick-Reference Summary Table

Vuln	OWASP 2023	CWE	Typical CVSS	Root Fix
Mass Assignment / Excessive Exposure	API3 (BOPLA)	CWE-915, CWE-213	7.5–9.8	DTO allow-list, separate read/write serializers
BFLA	API5	CWE-285/862/863	8.5–9.8	Default-deny RBAC middleware, per-endpoint check
Unrestricted Resource Consumption	API4	CWE-770/799/307	7.5–8.8	Rate limit + lockout + payload caps
Improper Inventory Mgmt	API9	CWE-1059	Varies (inherits underlying vuln)	API gateway single enforcement point, lifecycle policy
Security Misconfiguration	API8	CWE-16/209/548	4.0–9.8	Env-specific hardening gate, centralized error handler

30-second interview answer template: "REST API vulns beyond IDOR/injection mostly stem from authorization enforced inconsistently across object *properties* (BOPLA), *functions* (BFLA), and *inventory* (shadow/zombie APIs) — plus missing resource governance (rate limiting) and config hygiene. Root fix pattern is consistent: move from ad-hoc per-endpoint checks to centralized, default-deny enforcement (gateway/middleware) backed by schema-driven allow-lists."